



Universidad
Nacional
de Loja

Universidad Nacional de Loja

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS, Y LOS RECURSOS NATURALES NO RENOVABLES

Carrera de Ingeniería en Computación

ASIGNATURA:

Matemáticas discretas

UNIDAD:

Ciclo 1 "A"

TEMA:

Trabajo Autónomo: Ensayo y Aplicaciones de Estructuras de Árboles

DOCENTE:

Ing. Mario Enrique Cueva Hurtado

AUTORES:

Jhoan Alexander Rojas Guaman

Matias Joel Ortega Correa

Tyrone Efren Encarnación Erique

Erick Emanuel Andrade Montero

Introducción

Las Matemáticas Discretas constituyen una base fundamental para la informática, ya que proporcionan los conceptos necesarios para modelar y resolver problemas relacionados con la organización y el procesamiento de la información. Dentro de esta disciplina, la teoría de árboles ocupa un lugar importante debido a que permite representar estructuras jerárquicas de manera eficiente mediante grafos conexos y libres de ciclos. Estas características facilitan el diseño de algoritmos utilizados en sistemas de archivos, bases de datos, compiladores y motores de búsqueda (Villalpando Becerra, 2014).

La importancia de los árboles radica en que su organización influye directamente en el rendimiento de los algoritmos. Aspectos como la altura del árbol, la distribución de sus nodos y el tipo de recorrido determinan el tiempo necesario para localizar, insertar o procesar información. Por esta razón, comprender estas estructuras resulta indispensable para el desarrollo de soluciones computacionales eficientes.

El presente ensayo analiza los fundamentos de la teoría de árboles, sus principales tipos, la optimización mediante códigos Huffman, el funcionamiento de los árboles binarios de búsqueda y los recorridos Preorden, Inorden y Postorden. Asimismo, se presentan ejercicios prácticos que permiten aplicar estos conceptos y comprender su importancia dentro de las estructuras de datos utilizadas en la informática actual.

Desarrollo

Parte A. Fundamentos y Tipos de Árboles

La teoría de árboles constituye una extensión de la teoría de grafos y representa una de las estructuras más utilizadas para modelar relaciones jerárquicas. Un árbol se define como un grafo conexo que no contiene ciclos, condición que garantiza la existencia de un único camino entre dos vértices. Esta propiedad simplifica numerosos algoritmos de búsqueda y organización de información, razón por la cual los árboles tienen una amplia aplicación en la informática (Villalpando Becerra, 2014).

Una característica fundamental de los árboles es que un árbol con n vértices posee exactamente $n - 1$ aristas. Esta relación demuestra que se trata de una estructura mínima de conexión: si se elimina una arista, el árbol deja de ser conexo; si se agrega una nueva, aparece un ciclo. Desde el punto de vista computacional, estas propiedades permiten representar información de forma organizada y evitar recorridos redundantes.

Los árboles enraizados incorporan una organización jerárquica mediante un nodo principal denominado **raíz**, del cual se desprenden los demás nodos. Esta estructura permite definir

relaciones como padre, hijo, hermano y hoja, utilizadas para representar sistemas de archivos, árboles genealógicos y otras estructuras donde existe una dependencia entre los elementos.

Entre los principales tipos de árboles destacan los árboles generales, los árboles binarios y los árboles binarios de búsqueda. Mientras los árboles generales permiten un número variable de hijos por nodo, los árboles binarios limitan esa cantidad a dos, facilitando el desarrollo de algoritmos eficientes. Por su parte, los árboles binarios de búsqueda organizan la información según un criterio de orden, permitiendo realizar búsquedas más rápidas cuando la estructura permanece balanceada.

Las aplicaciones de la teoría de árboles abarcan numerosos campos de la informática. Se utilizan en sistemas operativos para organizar directorios, en bases de datos para acelerar consultas, en compiladores para analizar código fuente y en algoritmos de inteligencia artificial para representar procesos de decisión. Estas aplicaciones evidencian que la organización de la información es un factor determinante para el rendimiento de los sistemas computacionales.

Parte B. Optimización y Búsqueda

La eficiencia de un árbol depende en gran medida de su organización. Un concepto importante es la **longitud de paseo**, que representa el recorrido necesario para desplazarse entre los diferentes nodos de la estructura. Este concepto se relaciona con la altura del árbol, ya que un árbol más alto requiere un mayor número de comparaciones durante los procesos de búsqueda. En consecuencia, mantener árboles balanceados permite reducir el tiempo de acceso a la información (Villalpando Becerra, 2014).

Una de las aplicaciones más importantes de la teoría de árboles corresponde a los **códigos de prefijos o códigos Huffman**. Este algoritmo asigna códigos binarios más cortos a los símbolos de mayor frecuencia y códigos más largos a aquellos menos frecuentes, logrando una compresión eficiente de los datos. Su construcción se basa en un árbol binario formado a partir de las frecuencias de aparición de los símbolos, garantizando que ningún código sea prefijo de otro y evitando ambigüedades durante la decodificación. Gracias a estas características, Huffman continúa utilizándose en diversos formatos de compresión digital.

Los **Árboles Binarios de Búsqueda (ABB)** organizan los datos siguiendo un criterio de orden: los valores menores se ubican en el subárbol izquierdo y los mayores en el derecho. Esta disposición permite reducir progresivamente el espacio de búsqueda, ofreciendo un rendimiento cercano a $O(\log n)$ cuando el árbol permanece balanceado. Sin embargo, si la estructura pierde el equilibrio, la eficiencia disminuye y las operaciones pueden llegar a comportarse como una búsqueda lineal. Por ello, la correcta organización de los nodos resulta esencial para aprovechar las ventajas de esta estructura.

En conjunto, la longitud de paseo, los códigos Huffman y los árboles binarios de búsqueda demuestran que la teoría de árboles no solo organiza información, sino que también optimiza el uso del tiempo y del espacio en los sistemas informáticos.

Parte C. Recorridos de Árboles Binarios

Fundamento de los recorridos en árboles binarios

Una de las operaciones más importantes que puede realizarse sobre un árbol binario consiste en recorrer sistemáticamente todos sus nodos. Un recorrido es un procedimiento que establece el orden en que cada nodo será visitado exactamente una vez. Aunque todos los recorridos tienen como finalidad acceder a la totalidad de la información almacenada, cada uno responde a una lógica diferente y, por tanto, posee aplicaciones particulares dentro de la informática.

De acuerdo con Villalpando Becerra (2014), los recorridos fundamentales de un árbol binario son **Preorden**, **Inorden** y **Postorden**. La diferencia entre ellos radica en el momento en que se procesa la raíz respecto a sus subárboles izquierdo y derecho. Esta decisión aparentemente simple modifica completamente la secuencia obtenida y determina el tipo de problema para el cual resulta más conveniente cada recorrido.

En el recorrido **Preorden**, la raíz se procesa antes de visitar cualquiera de sus descendientes. En el recorrido **Inorden**, la raíz se visita únicamente después de haber recorrido el subárbol izquierdo y antes de continuar con el derecho. Finalmente, en el recorrido **Postorden**, la raíz constituye el último nodo visitado, una vez recorridos completamente ambos subárboles. Esta organización permite adaptar el recorrido a diferentes necesidades, como reconstrucción de estructuras, obtención de datos ordenados o liberación de memoria.

Desarrollo del APE fase tres:

FASE 3: Árboles y Estructuras de Datos Arbóreas (Semana 15: 13-17 julio 2026)

a. Los estudiantes construyen árboles binarios de búsqueda (BST) paso a paso, insertando valores numéricos y realizando los tres tipos de recorridos, documentando el orden de visita de nodos en cada caso.

Introducción

A continuación, se construyen tres BST distintos a partir de conjuntos de valores numéricos diferentes. Cada árbol se documenta insertando un valor a la vez y mostrando el gráfico del árbol resultante después de cada inserción. Al final de cada árbol se presentan los tres recorridos clásicos: inorden, preorden y postorden.

Reglas de inserción utilizadas

- Si el árbol está vacío, el valor insertado se convierte en la raíz.
- Si el valor es menor que el nodo actual, se compara con el subárbol izquierdo.
- Si el valor es mayor que el nodo actual, se compara con el subárbol derecho.
- El proceso se repite hasta encontrar una posición vacía (hoja) donde insertar el nuevo nodo.

Definición de los recorridos

- Inorden (Izquierda – Raíz – Derecha): en un BST produce los valores en orden ascendente.
- Preorden (Raíz – Izquierda – Derecha): útil para copiar o serializar el árbol.
- Postorden (Izquierda – Derecha – Raíz): útil para eliminar el árbol o evaluar expresiones.

Árbol Binario de Búsqueda N.º 1

Se insertará la siguiente secuencia de valores, uno a la vez:

50, 30, 70, 20, 40, 60, 80

Paso 1 — Insertar 50

Es el primer valor, se convierte en la raíz.

50

Paso 2 — Insertar 30

$30 < 50 \rightarrow$ va a la izquierda de 50.

50

/

30

Paso 3 — Insertar 70

$70 > 50 \rightarrow$ va a la derecha de 50.

```
  50_
 /  \
30  70
```

Paso 4 — Insertar 20

$20 < 50 \rightarrow$ izquierda; $20 < 30 \rightarrow$ izquierda de 30.

```
  50_
 /  \
30  70
/
20
```

Paso 5 — Insertar 40

$40 < 50 \rightarrow$ izquierda; $40 > 30 \rightarrow$ derecha de 30.

```
__50_
 /  \
30_  70
/  \
20  40
```

Paso 6 — Insertar 60

$60 > 50 \rightarrow$ derecha; $60 < 70 \rightarrow$ izquierda de 70.

```
__50__
 /    \
30_    70
/  \  /
```

20 40 60

Paso 7 — Insertar 80

$80 > 50 \rightarrow$ derecha; $80 > 70 \rightarrow$ derecha de 70.

```
  _50_
 /   \
30_   70_
/ \   / \
20 40 60 80
```

20 40 60 80

Recorridos del Árbol N.º 1

Recorrido	Orden de visita de nodos
Inorden (I-R-D)	20 - 30 - 40 - 50 - 60 - 70 - 80
Preorden (R-I-D)	50 - 30 - 20 - 40 - 70 - 60 - 80
Postorden (I-D-R)	20 - 40 - 30 - 60 - 80 - 70 - 50

Árbol Binario de Búsqueda N.º 2

Se insertará la siguiente secuencia de valores, uno a la vez:

15, 10, 20, 8, 12, 17, 25, 6

Paso 1 — Insertar 15

Es el primer valor, se convierte en la raíz.

15

Paso 2 — Insertar 10

$10 < 15 \rightarrow$ va a la izquierda de 15.

15

/

10

Paso 3 — Insertar 20

$20 > 15 \rightarrow$ va a la derecha de 15.

15_
/
10 20

Paso 4 — Insertar 8

$8 < 15 \rightarrow$ izquierda; $8 < 10 \rightarrow$ izquierda de 10.

15_
/
10 20
/
8

Paso 5 — Insertar 12

$12 < 15 \rightarrow$ izquierda; $12 > 10 \rightarrow$ derecha de 10.

__15_
/
10_ 20
/
8 12

Paso 6 — Insertar 17

$17 > 15 \rightarrow$ derecha; $17 < 20 \rightarrow$ izquierda de 20.

__15__
/ \

10_ 20

/ \ /

Recorrido	Orden de visita de nodos
Inorden (I-R-D)	6 - 8 - 10 - 12 - 15 - 17 - 20 - 25
Preorden (R-I-D)	15 - 10 - 8 - 6 - 12 - 20 - 17 - 25
Postorden (I-D-R)	6 - 8 - 12 - 10 - 17 - 25 - 20 - 15

8 12 17

Paso 7 — Insertar 25

$25 > 15 \rightarrow$ derecha; $25 > 20 \rightarrow$ derecha de 20.

__15__

/ \

10_ 20_

/ \ / \

8 12 17 25

Paso 8 — Insertar 6

$6 < 15 \rightarrow$ izquierda; $6 < 10 \rightarrow$ izquierda; $6 < 8 \rightarrow$ izquierda de 8.

__15__

/ \

10_ 20_

/ \ / \

8 12 17 25

/

Recorridos del Árbol N.º 2

Árbol Binario de Búsqueda N.º 3

Se insertará la siguiente secuencia de valores, uno a la vez:

45, 25, 65, 10, 35, 55, 75, 5, 15, 30, 40

Paso 1 — Insertar 45

Es el primer valor, se convierte en la raíz.

45

Paso 2 — Insertar 25

$25 < 45 \rightarrow$ va a la izquierda de 45.

45

/

25

Paso 3 — Insertar 65

$65 > 45 \rightarrow$ va a la derecha de 45.

45_

/ \

25 65

Paso 4 — Insertar 10

$10 < 45 \rightarrow$ izquierda; $10 < 25 \rightarrow$ izquierda de 25.

45_

/ \

25 65

/

10

Paso 5 — Insertar 35

$35 < 45 \rightarrow$ izquierda; $35 > 25 \rightarrow$ derecha de 25.

```
  __45__  
 /   \  
25_  65  
 /   \  
10 35
```

Paso 6 — Insertar 55

$55 > 45 \rightarrow$ derecha; $55 < 65 \rightarrow$ izquierda de 65.

```
  __45__  
 /   \  
25_  65  
 / \ /  
10 35 55
```

Paso 7 — Insertar 75

$75 > 45 \rightarrow$ derecha; $75 > 65 \rightarrow$ derecha de 65.

```
  __45__  
 /   \  
25_  65_  
 / \  / \  
10 35 55 75
```

Paso 8 — Insertar 5

$5 < 45 \rightarrow$ izquierda; $5 < 25 \rightarrow$ izquierda; $5 < 10 \rightarrow$ izquierda de 10.

```
  __45__  
 /   \  
25_  65_  
 / \  / \  
10 35 55 75
```

```

    25_   65_
   / \   / \
  10 35 55 75
 /
5

```

Paso 9 — Insertar 15

$15 < 45 \rightarrow$ izquierda; $15 < 25 \rightarrow$ izquierda; $15 > 10 \rightarrow$ derecha de 10.

```

    _45_
   /     \
  _25_   65_
 / \     / \
10_ 35 55 75
/ \
5 15

```

Paso 10 — Insertar 30

$30 < 45 \rightarrow$ izquierda; $30 > 25 \rightarrow$ derecha; $30 < 35 \rightarrow$ izquierda de 35.

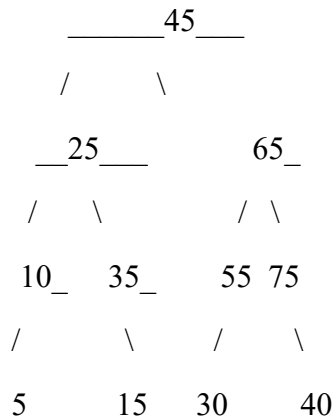
```

    _ _45_
   /       \
  _25_   _65_
 / \     / \
10_ 35   55 75
/       \   /
5      15 30

```

Paso 11 — Insertar 40

$40 < 45 \rightarrow$ izquierda; $40 > 25 \rightarrow$ derecha; $40 > 35 \rightarrow$ derecha de 35.



Recorridos del Árbol N.º 3

Recorrido	Orden de visita de nodos
Inorden (I-R-D)	5 - 10 - 15 - 25 - 30 - 35 - 40 - 45 - 55 - 65 - 75
Preorden (R-I-D)	45 - 25 - 10 - 5 - 15 - 35 - 30 - 40 - 65 - 55 - 75
Postorden (I-D-R)	5 - 15 - 10 - 30 - 40 - 35 - 25 - 55 - 75 - 65 - 45

Conclusiones

- En los tres árboles, el recorrido inorden produjo siempre los valores en orden ascendente, lo cual verifica que la propiedad fundamental del BST se respetó durante cada inserción.
- El orden de inserción determina la forma final del árbol: el Árbol N.º 2 mostró una rama más profunda por insertar valores de forma más secuencial, mientras que el Árbol N.º 3 resultó más equilibrado.
- El recorrido preorden resulta útil para reconstruir el árbol o copiarlo, ya que conserva el orden en que se insertó cada nodo respecto a su posición como raíz de cada subárbol.
- El recorrido postorden es apropiado para operaciones de eliminación, pues garantiza que los hijos se procesen antes que su nodo padre.

b. Los estudiantes investigan de forma autónoma sobre la relación entre la teoría de grafos/árboles y las estructuras de datos utilizadas en programación (heaps, tries, árboles B), bases de datos (índices) e inteligencia artificial (árboles de decisión). Elaboran un miniinforme con exposición apoyada en recursos TIC.

Teoría de grafos/árboles y las estructuras de datos utilizadas en programación

1. Introducción

Un grafo es un par (V, E) de vértices y aristas; un árbol es un caso particular: un grafo acíclico y conexo con una raíz y relaciones jerárquicas padre-hijo [1]. Esta abstracción matemática, formalizada desde el siglo XVIII, se traduce en programación como una estructura de datos: nodos enlazados por referencias o punteros que soportan operaciones eficientes de búsqueda, inserción y eliminación [1]. La transición no es trivial: mientras la teoría de grafos estudia propiedades abstractas (conectividad, ciclos, balance), la ingeniería de software prioriza la complejidad computacional (notación Big-O) y las restricciones físicas del hardware, como la memoria caché o el acceso a disco. De esa tensión entre teoría pura e implementación surgen estructuras especializadas —heaps, tries, árboles B, árboles de decisión— cada una optimizada para un problema concreto de programación, bases de datos o inteligencia artificial.

2. Núcleo de la investigación

A. Programación avanzada: Heaps, Tries y Árboles B

Es un árbol binario casi completo que cumple la propiedad de orden: cada nodo padre es mayor (max-heap) o menor (min-heap) que sus hijos [2]. Se implementa sobre un arreglo simple, evitando punteros explícitos. Su propósito de optimización clave es dar acceso $O(1)$ al elemento extremo y soportar inserción/extracción en $O(\log n)$, lo que lo convierte en la base de las colas de prioridad y del algoritmo Heapsort [2].

- Pros: acceso inmediato al máximo/mínimo; ordenamiento in-place sin memoria extra.

- Contrás: buscar un elemento arbitrario cuesta $O(n)$; no mantiene el arreglo totalmente ordenado.
- Casos de uso: colas de prioridad, algoritmo de Dijkstra, planificadores de sistemas operativos.

Organiza claves —típicamente cadenas de texto— de modo que cada nodo representa un carácter y cada camino raíz-nodo un prefijo compartido [3]. A diferencia de un árbol binario de búsqueda, es un árbol n -ario cuyo factor de ramificación depende del tamaño del alfabeto y no del número de claves almacenadas.

- Pros: búsqueda en $O(m)$, con m = longitud de la clave, independiente del total de elementos; soporta búsquedas por prefijo de forma natural.
- Contrás: alto consumo de memoria por los punteros de cada nodo (mitigable con tries comprimidos o radix trees).
- Casos de uso: autocompletado, correctores ortográficos, tablas de enrutamiento IP.

Generalizan el árbol binario de búsqueda permitiendo múltiples claves e hijos por nodo, y se mantienen siempre balanceados por construcción [4]. Su objetivo central es minimizar la altura del árbol y, con ella, el número de accesos a memoria secundaria, razón por la que dominan el diseño de índices de bases de datos [4], [5] (desarrollado en la sección B).

B. Bases de datos: índices B-Tree / B+-Tree

Un índice de base de datos es, en esencia, un árbol B o B+ que actúa como mapa ordenado entre valores de columna y ubicaciones físicas de los registros [4]. Al tratarse de un grafo acíclico balanceado con factor de ramificación k , la profundidad del árbol crece en $O(\log_k n)$, por lo que localizar un registro entre millones requiere solo unos pocos accesos a disco [5]. La diferencia entre B-Tree y B+-Tree es estructural: en un B+-Tree los datos residen únicamente en las hojas, y estas se enlazan secuencialmente entre sí, lo que optimiza tanto la búsqueda puntual como los recorridos por rango (ORDER BY, BETWEEN) [4], [6]. Motores como InnoDB (MySQL) o PostgreSQL emplean variantes de B+-Tree como estructura por defecto para sus índices [6].

Estructura	Datos en nodos internos	Búsqueda por rango	Complejidad de búsqueda	Caso de uso típico
B-Tree	Sí	Moderada	$O(\log n)$	Índices generales, sistemas de archivos

Estructura	Datos en nodos internos	Búsqueda por rango	Complejidad de búsqueda	Caso de uso típico
B+-Tree	No (solo en hojas enlazadas)	Excelente	$O(\log n)$	Índices de motores relacionales (MySQL, PostgreSQL)
Índice Hash	No aplica (tabla hash)	Nula	$O(1)$ promedio	Búsquedas de igualdad exacta (key-value stores)

C. Inteligencia artificial: árboles de decisión

Un árbol de decisión modela el aprendizaje supervisado como una secuencia de particiones jerárquicas del espacio de atributos: cada nodo interno aplica una prueba sobre una variable, cada rama representa un resultado posible y cada hoja asigna una clase o valor [7]. Algoritmos como ID3 y su sucesor C4.5 construyen el árbol de forma recursiva (top-down induction), seleccionando en cada nodo el atributo que maximiza la ganancia de información, es decir, la mayor reducción de entropía [7].

- Pros: alta interpretabilidad, ya que puede leerse como un conjunto de reglas si-entonces; no requiere normalizar los datos; admite variables categóricas y numéricas.
- Contras: tiende al sobreajuste (overfitting) en árboles profundos y presenta alta varianza ante pequeños cambios en los datos de entrenamiento.

Para reducir la varianza, métodos como Random Forest combinan cientos de árboles entrenados sobre subconjuntos aleatorios de datos y variables, promediando sus predicciones y mejorando la generalización respecto de un único árbol [8].

- Casos de uso: diagnóstico médico asistido, detección de fraude, sistemas de recomendación, motores de decisión en robótica.

Portafolios de los integrantes:

Jhoan:

<https://github.com/alexjhoan07-wq/Matem-ticas-Discretas/blob/main/Portafolio.md>

Tyrone:

<https://github.com/tyroneencar8767-ship-it/portafolioDigitalMD/blob/main/README.md>

Matias:

<https://matiasoc10.github.io/PortafolioMatem-ticasDiscretas/>

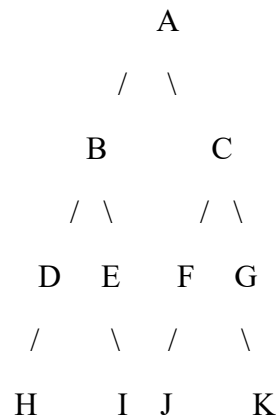
Erick:

https://github.com/Blooml2/portafolio-de-matematicas/tree/main/mat_discretas

Ejercicios resueltos:

Ejercicio 1

Árbol binario



Este árbol posee cuatro niveles de profundidad y será utilizado para aplicar los tres recorridos fundamentales.

Recorrido Preorden

Regla:

Raíz → Subárbol izquierdo → Subárbol derecho

Desarrollo paso a paso

Paso 1. Se inicia en la raíz del árbol, por lo que el primer nodo visitado es **A**.

Paso 2. Se desciende al subárbol izquierdo y se visita **B**.

Paso 3. Desde B se continúa hacia su hijo izquierdo, visitando **D**.

Paso 4. Se visita el hijo izquierdo de D, correspondiente a **H**. Como H es una hoja, se regresa a D.

Paso 5. Se visita el hijo derecho de D, correspondiente a **I**.

Paso 6. Finalizado el subárbol de D, se regresa a B y se visita su hijo derecho, **E**.

Paso 7. Desde E se visitan primero **J** y luego **K**.

Paso 8. Una vez recorrido completamente el subárbol izquierdo, se vuelve a la raíz y se continúa con el subárbol derecho, visitando **C**, luego **F** y finalmente **G**.

Resultado

A → B → D → H → I → E → J → K → C → F → G

Análisis

En este recorrido la raíz siempre aparece antes que sus descendientes. Esta característica permite reconstruir la estructura completa del árbol cuando se combina con otros recorridos y es ampliamente utilizada para copiar árboles binarios.

Recorrido Inorden

Regla:

Subárbol izquierdo → Raíz → Subárbol derecho

Desarrollo paso a paso

1. Se inicia recorriendo completamente el subárbol izquierdo de A.
2. Dentro de ese subárbol se visita primero H.

3. Luego se procesa D.
4. Después se visita I.
5. Se regresa a B.
6. Se recorren J, E y K.
7. Finalizado el subárbol izquierdo, se procesa la raíz A.
8. Finalmente se recorren F, C y G.

Resultado

$H \rightarrow D \rightarrow I \rightarrow B \rightarrow J \rightarrow E \rightarrow K \rightarrow A \rightarrow F \rightarrow C \rightarrow G$

Análisis

Cuando el árbol corresponde a un Árbol Binario de Búsqueda, este recorrido produce automáticamente los elementos ordenados de menor a mayor.

Recorrido Postorden

Regla:

Subárbol izquierdo $\rightarrow$ Subárbol derecho $\rightarrow$ Raíz

Desarrollo paso a paso

Primero se recorren completamente los hijos de D (H e I), después D; luego los hijos de E (J y K), después E; posteriormente B. Se continúa con F y G antes de visitar C. Finalmente se procesa la raíz A.

Resultado

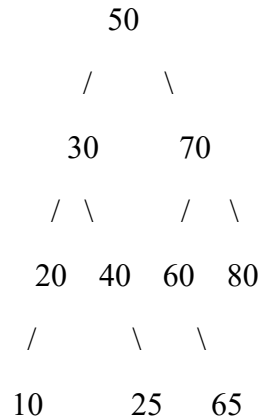
$H \rightarrow I \rightarrow D \rightarrow J \rightarrow K \rightarrow E \rightarrow B \rightarrow F \rightarrow G \rightarrow C \rightarrow A$

Análisis

Este recorrido deja la raíz para el final, razón por la cual se utiliza frecuentemente para eliminar árboles de memoria o evaluar expresiones representadas mediante árboles.

Ejercicio 2

Árbol Binario de Búsqueda



Desarrollo del recorrido Preorden

1. Se visita primero la raíz (**50**).
2. Se continúa con el subárbol izquierdo (**30**).
3. Luego se visitan **20**, **10** y **25**.
4. Se regresa para visitar **40**.
5. Finalmente se recorre el subárbol derecho: **70**, **60**, **65** y **80**.

Resultado

50 → 30 → 20 → 10 → 25 → 40 → 70 → 60 → 65 → 80

Desarrollo del recorrido Inorden

Se recorre completamente el lado izquierdo antes de visitar cada raíz.

Resultado

10 → 20 → 25 → 30 → 40 → 50 → 60 → 65 → 70 → 80

Análisis

En este ejercicio puede comprobarse que el recorrido Inorden devuelve todos los valores en orden ascendente. Esto no ocurre por casualidad, sino porque el Árbol Binario de Búsqueda mantiene la propiedad de almacenar todos los valores menores a la izquierda y los mayores a la derecha de cada nodo.

Desarrollo del recorrido Postorden

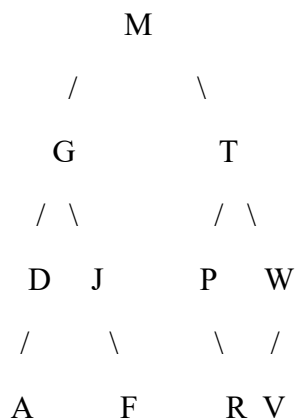
Se recorren primero ambos subárboles y únicamente después se visita cada raíz.

Resultado

10 → 25 → 20 → 40 → 30 → 65 → 60 → 80 → 70 → 50

Ejercicio 3

Árbol binario



Recorrido Preorden

Desarrollo

La raíz M se visita primero. Posteriormente se recorre completamente el subárbol izquierdo (G, D, A, F y J). Finalmente se visita el subárbol derecho (T, P, R, W y V).

Resultado

M → G → D → A → F → J → T → P → R → W → V

Recorrido Inorden

Resultado

A → D → F → G → J → M → P → R → T → V → W

Análisis

La secuencia obtenida demuestra que el recorrido Inorden respeta el criterio de orden del árbol, lo cual explica su utilidad en los árboles binarios de búsqueda para listar información ordenada.

Recorrido Postorden

Resultado

A → F → D → J → G → R → P → V → W → T → M

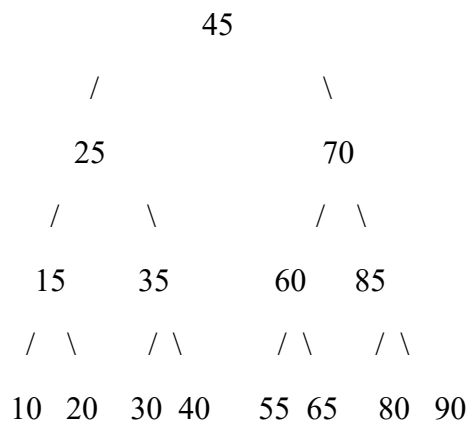
Comentario final del ejercicio

Aunque los tres recorridos visitan exactamente los mismos nodos, el orden cambia completamente debido a la posición que ocupa la raíz dentro del algoritmo. Esta diferencia determina la aplicación práctica de cada recorrido en distintos problemas informáticos.

Parte C (Continuación). Recorridos de Árboles Binarios

Ejercicio 4

Árbol Binario de Búsqueda



Este árbol posee cuatro niveles de profundidad y mantiene la propiedad fundamental de un Árbol Binario de Búsqueda: todos los valores menores que un nodo se encuentran a su izquierda y los mayores a su derecha.

Recorrido Preorden

Regla: Raíz → Izquierdo → Derecho.

Desarrollo paso a paso

Paso 1. Se inicia el recorrido en la raíz, por lo que el primer nodo visitado es **45**.

Paso 2. Se continúa con el subárbol izquierdo visitando **25**.

Paso 3. Desde 25 se recorre el nodo **15**, seguido de sus hijos **10** y **20**.

Paso 4. Se regresa a 25 para recorrer su hijo derecho (**35**) y posteriormente sus descendientes **30** y **40**.

Paso 5. Finalizado el subárbol izquierdo, se vuelve a la raíz (**45**) y se inicia el recorrido del subárbol derecho.

Paso 6. Se visitan sucesivamente los nodos **70, 60, 55, 65, 85, 80** y **90**.

Resultado

45 → 25 → 15 → 10 → 20 → 35 → 30 → 40 → 70 → 60 → 55 → 65 → 85 → 80 → 90

Análisis

El recorrido Preorden siempre procesa primero la raíz de cada subárbol. Esta característica permite reconstruir la estructura original del árbol cuando se combina con otro tipo de recorrido, razón por la cual es ampliamente utilizado por compiladores y algoritmos de serialización de datos.

Recorrido Inorden

Regla: Izquierdo → Raíz → Derecho.

Desarrollo paso a paso

1. Se recorre completamente el subárbol izquierdo de la raíz.
2. Dentro del nodo 25 se visitan primero los valores 10, 15 y 20.
3. Luego se procesa el nodo 25.
4. Después se visitan 30, 35 y 40.
5. Finalizado el lado izquierdo, se procesa la raíz 45.
6. Finalmente se recorren los nodos del subárbol derecho siguiendo el mismo procedimiento.

Resultado

10 → 15 → 20 → 25 → 30 → 35 → 40 → 45 → 55 → 60 → 65 → 70 → 80 → 85 → 90

Análisis

El resultado confirma una de las propiedades más importantes de los Árboles Binarios de Búsqueda: el recorrido Inorden genera automáticamente una secuencia ordenada de menor a

mayor. Esta propiedad explica su uso frecuente en sistemas que requieren ordenar información sin aplicar algoritmos adicionales de clasificación.

Recorrido Postorden

Regla: Izquierdo → Derecho → Raíz.

Desarrollo paso a paso

Se recorren primero todos los descendientes izquierdos, luego los derechos y únicamente al finalizar se procesa cada nodo padre. La raíz del árbol se visita al final del recorrido.

Resultado

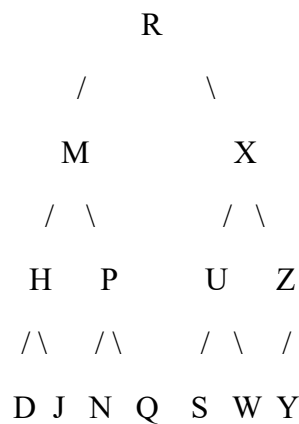
10 → 20 → 15 → 30 → 40 → 35 → 25 → 55 → 65 → 60 → 80 → 90 → 85 → 70 → 45

Análisis

El recorrido Postorden resulta especialmente útil cuando es necesario eliminar una estructura de datos de manera segura, ya que primero procesa todos los descendientes y únicamente al final elimina el nodo padre.

Ejercicio 5

Árbol Binario



Este árbol representa una estructura jerárquica organizada mediante caracteres alfabéticos. Aunque no corresponde necesariamente a un ABB perfecto, permite analizar claramente las diferencias entre los tres recorridos fundamentales.

Recorrido Preorden

Desarrollo paso a paso

El recorrido comienza visitando la raíz **R**. Posteriormente se procesa el subárbol izquierdo recorriendo los nodos **M, H, D, J, P, N** y **Q**. Finalmente se visita el subárbol derecho siguiendo el orden **X, U, S, W, Z** y **Y**.

Resultado

$R \rightarrow M \rightarrow H \rightarrow D \rightarrow J \rightarrow P \rightarrow N \rightarrow Q \rightarrow X \rightarrow U \rightarrow S \rightarrow W \rightarrow Z \rightarrow Y$

Recorrido Inorden

Desarrollo paso a paso

Se recorren primero todos los nodos del subárbol izquierdo antes de visitar cada raíz. Después de procesar la raíz principal (**R**), se continúa aplicando el mismo procedimiento al subárbol derecho.

Resultado

$D \rightarrow H \rightarrow J \rightarrow M \rightarrow N \rightarrow P \rightarrow Q \rightarrow R \rightarrow S \rightarrow U \rightarrow W \rightarrow X \rightarrow Y \rightarrow Z$

Análisis

Este recorrido evidencia cómo la posición relativa de cada nodo depende de la organización interna del árbol. Cuando la estructura cumple las propiedades de un ABB, la secuencia obtenida aparece ordenada automáticamente; en otros árboles binarios, el recorrido mantiene el orden estructural definido por sus ramas.

Recorrido Postorden

Desarrollo paso a paso

En este recorrido se procesan completamente ambos subárboles antes de visitar cada nodo padre. Como consecuencia, la raíz **R** constituye el último elemento de la secuencia.

Resultado

$D \rightarrow J \rightarrow H \rightarrow N \rightarrow Q \rightarrow P \rightarrow M \rightarrow S \rightarrow W \rightarrow U \rightarrow Y \rightarrow Z \rightarrow X \rightarrow R$

Comparación Analítica de los Recorridos

Los ejercicios desarrollados permiten comprobar que los recorridos Preorden, Inorden y Postorden no constituyen simplemente diferentes formas de visitar un árbol, sino estrategias diseñadas para resolver problemas específicos.

El recorrido **Preorden** procesa primero la raíz de cada subárbol, por lo que resulta adecuado para reconstruir estructuras jerárquicas, copiar árboles y representar expresiones.

El recorrido **Inorden** adquiere especial importancia en los Árboles Binarios de Búsqueda porque aprovecha la relación de orden entre los nodos para generar automáticamente una secuencia ordenada. Esta propiedad evita aplicar algoritmos adicionales de clasificación y mejora la eficiencia del procesamiento de datos.

Por su parte, el recorrido **Postorden** retrasa el procesamiento de la raíz hasta haber recorrido completamente sus descendientes. Esta estrategia resulta especialmente útil cuando las operaciones sobre un nodo dependen previamente de la información contenida en sus hijos, como ocurre durante la eliminación de árboles o la evaluación de expresiones matemáticas.

En conjunto, los tres recorridos demuestran que una misma estructura puede analizarse desde perspectivas diferentes dependiendo del objetivo del algoritmo. La elección del recorrido adecuado influye directamente en la eficiencia y en la correcta resolución del problema computacional planteado.

Conclusiones

La teoría de árboles constituye una herramienta esencial dentro de las Matemáticas Discretas debido a que permite representar relaciones jerárquicas y desarrollar algoritmos eficientes para el procesamiento de la información.

El análisis realizado permitió comprobar que la eficiencia de estructuras como los árboles binarios de búsqueda depende directamente de su organización. Un árbol balanceado reduce el tiempo necesario para localizar datos, mientras que una estructura desbalanceada afecta significativamente su rendimiento.

Asimismo, el estudio de los códigos Huffman evidenció que los árboles también permiten optimizar el almacenamiento de información mediante técnicas de compresión que continúan siendo utilizadas en aplicaciones actuales.

Finalmente, los ejercicios desarrollados demostraron que los recorridos Preorden, Inorden y Postorden responden a objetivos diferentes y que la selección del recorrido adecuado depende del problema que se desea resolver. Esto confirma la importancia de comprender tanto la teoría como su aplicación práctica dentro del desarrollo de software y las estructuras de datos.

Reflexiones Personales

Reflexión de Jhoan:

La elaboración de este ensayo fortaleció mis habilidades para investigar, analizar información técnica y relacionar los conceptos matemáticos con aplicaciones reales. Entender el funcionamiento de los árboles binarios y de los diferentes recorridos me permitió apreciar cómo las Matemáticas Discretas sirven como base para el desarrollo de tecnologías utilizadas diariamente. Considero que estos conocimientos serán de gran utilidad durante mi formación académica y en mi futura práctica profesional.

Reflexión de Matías:

La investigación realizada me permitió reconocer la importancia de las estructuras de datos dentro de la informática moderna. Comprendí que un algoritmo eficiente depende no solo de su diseño, sino también de la forma en que la información se organiza. Este aprendizaje me motiva a seguir profundizando en temas relacionados con programación y análisis de algoritmos, ya que constituyen competencias fundamentales para el ejercicio profesional.

Reflexión de Tyrone:

Este trabajo me permitió comprender que las Matemáticas Discretas tienen una aplicación práctica mucho más amplia de lo que imaginaba al inicio del curso. El estudio de los árboles y sus recorridos me ayudó a entender cómo muchas herramientas informáticas organizan la información de forma eficiente. Además, desarrollar los ejercicios fortaleció mi capacidad de análisis lógico y de resolución de problemas, habilidades que considero indispensables para mi formación profesional.

Reflexión de Erick:

Durante la elaboración de este trabajo comprendí que la teoría de árboles constituye una herramienta esencial para el desarrollo de soluciones informáticas eficientes. Analizar la relación entre la organización de los datos y el rendimiento de los algoritmos me permitió valorar la importancia de seleccionar estructuras adecuadas según las necesidades de cada problema. Considero que estos conocimientos fortalecerán mi formación profesional, especialmente en asignaturas relacionadas con estructuras de datos, bases de datos y desarrollo de software, donde la eficiencia de los algoritmos es un aspecto fundamental.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Java* (6th ed.). Wiley.

Rosen, K. H. (2019). *Matemáticas discretas y sus aplicaciones* (8.^a ed.). McGraw-Hill Education.

Villalpando Becerra, J. F. (2014). *Matemáticas discretas: aplicaciones y ejercicios*. McGraw-Hill Education.